RiskRadar

From raw news to risk signals — an AI/NLP Risk Engine that rebalances an index and stress-tests a bank portfolio, live.

What this document is
The full explanation of my Phase III submission — what I built, why every decision was made, what results it produced, and a Q&A prep section so any question about the project can be answered from first principles.

The one-line pitch
“Markets move on news, but risk systems only eat numbers. RiskRadar converts real-time news and social media into three machine-actionable numbers — sentiment, event type, impact — and two modules act on them: a sentiment-driven index rebalancer and an event-driven portfolio stress tester.”

Submission · Phase III — Case Study Competition

Submitted by · Aare Renu Shri

Deliverables · source code (repo) · live demo (≤5 min) · this report · 7-slide deck

Runs · fully offline, one command: `python run.py` → dashboard at localhost:8000

✎ Add your roll ID (e.g., cs24btech11001) here before printing.

1 The 60-second summary

Everything below is expanded in later sections; this page is the whole story.

CORE ENGINE

AI/NLP Risk Engine

Reads **two source types** (financial news + social media), and for every item outputs a structured signal: **sentiment (–1…+1)**, **event classification** (12 categories), **impact score (1–10)** — each one *explainable* (keywords & drivers stored on the signal).

MODULE A (BUILT)

Index Rebalancing

Subscribes to sentiment and tilts a 20-stock mock index (S&P 100 subset): `w × (1 + 0.25·s)`, clamped to 2–12%, renormalized. Live dashboard shows weights drifting over time.

MODULE B (BUILT)

Stress Testing

Subscribes to event + impact. Any event with **impact ≥ 7** triggers an event-typed shock scenario (e.g. rates +200bp, equity –10%) on a **$5.36bn synthetic wholesale portfolio** (38 positions, 9 asset classes), with before/after valuation and P&L per class.

How everything connects

Sources push text into the engine (a publish/subscribe core). The engine analyzes it and notifies every subscriber — both modules, the API, and persistence. The modules never talk to the sources; the engine doesn't know who listens. That one design choice is why adding Module B after Module A took ~150 lines and no refactoring.

Headline numbers (real, from my headless end-to-end run)

Metric	Value	Metric	Value
Items processed (news + social)	**58** (28 + 30)	Median latency per signal	**0.3 ms** (~3,300/sec, 1 core)
Meaningfully classified	**54 / 58 (93%)**, 4 "Other"	Module A weight changes	**45** (43 rebalancing signals)
High-risk flags (impact ≥7 & negative)	**22**	Module B stress tests triggered	**23** (threshold: impact ≥ 7)
Portfolio value	**$5.36bn**, 38 positions	Worst stress result	**–$435m (–8.1%)** rates shock

What to say if you get 30 seconds with a judge

> "I built one engine that turns any news or tweet into three explainable numbers — sentiment, event type, impact. Then I built *both* downstream options: a tactical module that rebalances a 20-stock index as sentiment arrives, and a strategic module that fires stress scenarios at a synthetic $5.36bn bank portfolio when high-impact events land. It all runs live, offline, on one laptop — type any headline into my dashboard and you'll watch it flow through the entire system in real time."

2 The problem, and how I interpreted it

The case study asks for a platform that ingests real-time unstructured data and produces actionable financial risk signals. Broken down, it has one mandatory core and (at least) one downstream module. I built the core **and both** modules.

Requirement → where it lives

Requirement	How I satisfy it	Where
Ingest text from **≥ 2 sources** (e.g. news + Twitter/X)	Two bundled feeds (financial news + social posts) streamed by a live-feed simulator; plus real connectors — **NewsAPI**, **GDELT**, **Reddit** — that activate when keys/internet are present (fail-soft: the demo never breaks)	`engine/ingest.py`
Sentiment score (–1.0 … +1.0)	VADER compound score, extended with a hand-curated finance lexicon (~150 terms: downgrade –2.6, buyback +2.6 …). VADER handles emoji/caps/negation natively — built for tweets	`engine/lexicon.py`
Event classification (categorical)	Priority-ordered keyword rules over 12 categories; every signal stores the keywords that fired (audit trail)	`engine/classifier.py`
Impact score (1–10)	Transparent formula: `base(category) ± urgency ± magnitude ± sentiment ± scope`, clamped; drivers recorded on each signal	`engine/impact.py`
Output consumable by downstream apps **(API or file)**	Both: REST API (`/api/signals` …) and files — `data/signals.jsonl` + SQLite history	`api/server.py`, `engine/store.py`
Module A — sentiment-driven rebalancing + weights dashboard	Multiplicative tilt k=0.25, 2–12% band, renormalize; audit log; live charts	`modules/rebalancer.py`
Module B — event-driven stress test on mixed portfolio + before/after dashboard	Loans + bonds + MBS + derivatives + commitments; scenario shocks per event type; before/after + P&L by class + worst positions	`modules/stress_test.py`

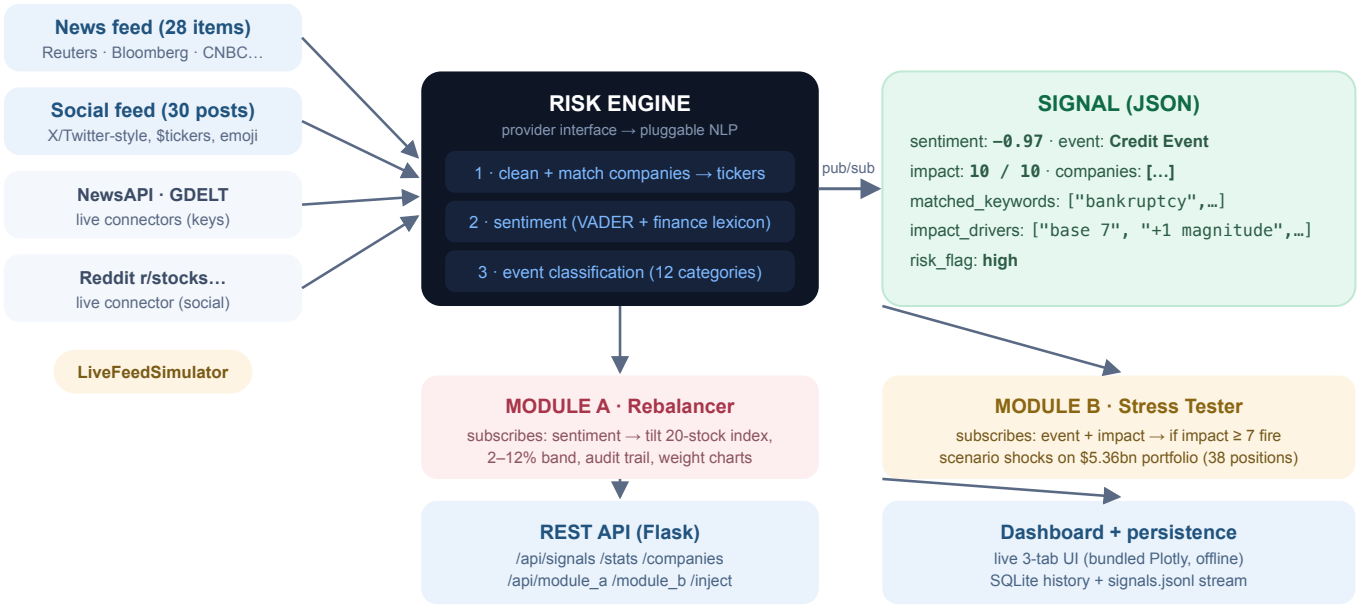
Scope honesty: the case study explicitly allows a simplified stress model and a mock index — I leaned into that: simple, defensible first-order models, with the *engineering* (streaming, subscriptions, fail-soft, explainability) done for real.

What "risk signal" means to me

Every signal carries more than the three required scores: companies mentioned (mapped to tickers), the keywords that drove classification, the drivers that drove impact, and a triage flag (`none` `watch` `high`) — the "why" was in the data model from day one.

3 Architecture

One process, one data flow direction, zero hidden magic.



Walkthrough (follow the arrows)

1 · Sources → engine. Every source is a background thread pushing *NewsItem* objects into the engine — the engine cannot tell the simulator from NewsAPI, which is exactly why the offline demo is indistinguishable from the live one. **2 · Engine → signal.** Clean text, match companies against a 20-name ticker dictionary, score sentiment, classify event, compute impact. **3 · Publish.** The engine stores the signal (SQLite + JSONL) and notifies every subscriber. **4 · Consumers act.** Module A rebalances if the signal touches an index member; Module B stress-tests if impact ≥ 7. The API serves everything to the dashboard.

Why this shape

- **Publish/subscribe core** — modules are three-line plug-ins; a third consumer (alerts? trading?) is one function call, no engine changes.
- **Provider interface** — the NLP sits behind one method, so upgrading VADER → FinBERT/LLM later is a one-file change, not a rewrite.
- **Fail-soft everywhere** — any connector can die; nothing downstream notices.

4 The Risk Engine, part 1 — ingestion & sentiment

Ingestion: two source types, minimum met — plus live connectors

The requirement was “at least two different sources (e.g. financial news articles and Twitter/X posts)”. I have two feed *types* (news + social), each with multiple concrete sources:

Source	Type	When active	Notes
Bundled news feed (28 items)	news	always	hand-written, realistic headlines spanning every event category (Reuters/Bloomberg/CNBC/FT/WSJ style)
Bundled social feed (30 posts)	social	always	X/Twitter-style posts: $tickers, emoji, slang, negation
LiveFeedSimulator	both	always (default)	streams the two feeds with jitter — the rest of the system cannot tell it apart from a real firehose
NewsAPI + GDELT DOC 2.0	news	key / internet	free tiers; GDELT needs no key (the case study’s own recommended dataset)
Reddit (r/wallstreetbets, r/stocks)	social	internet available	public JSON API, no key needed

Bundled items are labeled as sample data in the repo; live connectors are real code, fail-soft by design — no key or no network just logs “inactive”.

Sentiment: VADER, but taught finance

VADER (Valence Aware Dictionary and sEntiment Reasoner) computes a compound score in [–1, +1]. I chose it because **it was literally built for social media** — it understands emoji (“🚀 ”), ALL-CAPS boosting, intensifiers (“insane”) and negation (“not good”) natively, which matters when half your corpus is tweets. The catch: VADER’s dictionary doesn’t know finance jargon — “downgrade”, “contagion”, “buyback” would score as neutral. So I extended its lexicon with ~150 hand-curated finance terms:

```
"bankruptcy": -3.5   "downgrade": -2.6   "selloff": -2.6   "tariffs": -2.2   "recall": -2.6
"contagion": -2.8
"record": +2.4     "surge": +2.6     "buyback": +2.6   "breakthrough": +2.8 "beats": +2.4    "soft
landing": +2.0
```

Real examples from my run:

Text (abridged)	Sentiment	Why it worked
“Regional lender Pacific Crest Bank files for Chapter 11 bankruptcy… largest failure since 2023”	**–0.97**	bankruptcy + contagion + failure (all in my lexicon)
“Coca-Cola recall: contamination scare triggers nationwide product recall”	**–0.98**	recall + contamination + scare; negation-safe

Labels: ≥ +0.15 positive · ≤ –0.15 negative · else neutral (30 pos / 28 neg / 0 neu on the corpus).

Why not FinBERT or an LLM? (1) it must run live on any laptop, zero internet — a demo that can break is a failed demo; (2) every score must be explainable — I show the words behind “–0.97”; (3) the provider interface makes upgrading a one-file change (§10). VADER is my *v1 engine*, not my ceiling.

4 The Risk Engine, part 2 — classification & impact

Event classification: 12 categories, priority-ordered rules

Categories (ours extend the case study's list): Geopolitical · Macroeconomic · Credit Event · Merger/Acquisition · Product Launch · Regulatory/Legal · Earnings · Cybersecurity · Supply Chain · Operational/Quality · Market Volatility · Other

A rule = (category, priority, keyword list). Classification picks the category with the most keyword matches; ties resolved by priority. **Priority encodes domain knowledge** — the story "bank files for Chapter 11 amid a lawsuit" mentions both credit and legal words; priority says it's a **Credit Event**, because that's how a risk desk would file it. Classification results on my corpus: **54/58 meaningful (93%)**; the 4 "Other" items were genuinely casual chatter (e.g. "the macro picture is genuinely confusing right now") — correct behavior, not a miss: those items received low impact too.

Impact score: a formula you can audit, not a number you trust blindly

```
impact = clamp( base(category) + urgency + magnitude −/+ sentiment + scope , 1 , 10 )
```

Component	Values	Reasoning
base(category)	Credit 7 · Geo 6.5 · Macro 6 · Cyber 5.5 · M&A 5 · Legal 5 · Volatility 4.5 · Supply 4 · Operational 4.5 · Earnings 3.5 · Product 3 · Other 2.5	typical severity of the event class — a credit event moves markets more than a product launch
urgency	+1 if "breaking / surprise / escalating…"	fresh, unfolding events are riskier than analysis pieces
magnitude	+1 "record/massive/largest" · −1 "minor/slight"	scale words are cheap, reliable severity signals
sentiment	+1 if score ≤ −0.55 · −0.5 if ≥ +0.55	bad news hits harder than good news lifts (asymmetry is real in markets)
scope	+1 if ≥2 companies or "sector-wide/global"	contagion beats idiosyncrasy

Worked example — one real signal, end to end:

```
IN : "OPEC+ surprise production cut escalates trade tensions with Washington"
OUT : {
  "sentiment": -0.84,  "event_type": "Geopolitical",  "impact": 8.5,  "risk_flag": "high",
  "matched_keywords": ["opec","geopolitical","escalating","tensions","washington"],  // why this
class
  "impact_drivers": ["base 6.5 (Geopolitical)", "+1 urgency (breaking/surprise)", "+1 strongly
negative sentiment"]  // why 8.5
}
```

Also real: *"Visa & Mastercard face antitrust lawsuit"* → Regulatory/Legal, −0.92, impact **8.0**, flag **high**; *"Apple unveils iPhone 17 Pro"* → Product Launch, +0.74, impact **2.5**.

Output — API *and* files (the case study said "or"): `/api/signals` · `/api/stats` · `/api/module_a` · `/api/module_b` · `POST /api/inject` · **signals.jsonl** (grep-able stream) + **SQLite** (history for backtests).

5 Module A — tactical index rebalancing (sentiment-fed)

The tactical consumer: reacts to *every* sentiment signal that touches an index member.

THE REBALANCING RULE

```
w_new = w_old × (1 + k × sentiment)     //
k = 0.25
w      = clamp(w, 2%, 12%)              //
position limits
w     /= Σ w
// renormalize to 100%
```

WHY EACH PIECE

- **Multiplicative tilt** — positive sentiment must be earned repeatedly to grow a position; a single bad headline can at most shrink a weight by 25% of its size. Multiplicative = proportional, self-consistent.
- **k = 0.25** — one maximal good event (+1.0) lifts a stock 25%; meaningful, not jumpy.
- **2–12% band** — real indices enforce concentration caps; also stops runaway domination in my demo.
- **Renormalize** — it's a portfolio: weights must always sum to 100%.

Universe

20 large caps (a subset of the S&P 100 as the case study suggests), spread across sectors: AAPL, MSFT, NVDA, AMZN, GOOGL, META, TSLA, JPM, BAC, GS, XOM, CVX, JNJ, PFE, KO, MCD, WMT, DIS, BA, CAT. Starts equal-weighted at 5% each.

Real results from the headless run

	Ticker	Weight	What happened
Winners	MCD	7.4%	value-menu expansion + Asia rebound, repeatedly positive social + news
	WMT	7.3%	guidance raise, "boring best stock" sentiment
	JPM	6.6%	record profit, dividend + $30B buyback
Losers	KO	2.6%	nationwide contamination recall (impact 7.5, high flag)
	BAC	2.6%	subprime default warnings, +$1.2B provision
	BA	2.9%	supply-chain disruption into 2027

43 of the 58 signals touched an index member and drove **45 weight changes** (some signals mention two names, e.g. "Goldman Sachs and Morgan Stanley…"). The dashboard replays this as a live chart of weights drifting away from the equal-weight line — the money visual for Module A.

What I log (auditability)

Every rebalance records: ticker, sentiment, event type, impact, old weight → new weight, and the signal ID that caused it. Any decision can be reconstructed after the fact — the same standard a real risk system would be held to.

6 Module B — strategic stress testing (event-fed)

The strategic consumer: ignores daily chatter and wakes only for **high-impact events (impact ≥ 7)** — the case study's own threshold.

The synthetic portfolio (built to the case study's spec)

Asset class	Positions	Notional	Examples
Corporate loans	12	$1,625m	term loans, RCFs, leveraged loans, project finance, shipping, aircraft
CRE loans	3	$415m	office, logistics, retail mortgages
Government bonds	4	$950m	US 10Y/30Y, Bund, TIPS
Corporate bonds	7	$700m	IG + HY, incl. names the portfolio also lends to (concentration!)
Mortgage bonds	2	$500m	agency MBS/CMBS
Derivatives	6	$740m	pay/receive-fixed IRS, cross-currency swap, equity forward, sold CDS
Loan commitments	2	$180m	undrawn RCFs
Direct equity	2	$250m	S&P basket, PE fund LP
Total	**38**	**$5,360m**	real fields: rating, duration, spread, direction

Scenario shocks, keyed to the engine's event classification

Engine event	Scenario	Rate Δ	Credit spread	Equity	Loan haircut
Geopolitical	Geo shock	+50bp	+180bp	−12%	1%
Macroeconomic	Rates shock	**+200bp**	+60bp	−10%	1%
Credit Event	Credit shock	+100bp	**+250bp**	−8%	3%
Cybersecurity	Cyber incident	—	+40bp	−6%	0.5%
Regulatory/Legal	Regulatory action	—	+50bp	−6%	0.5%

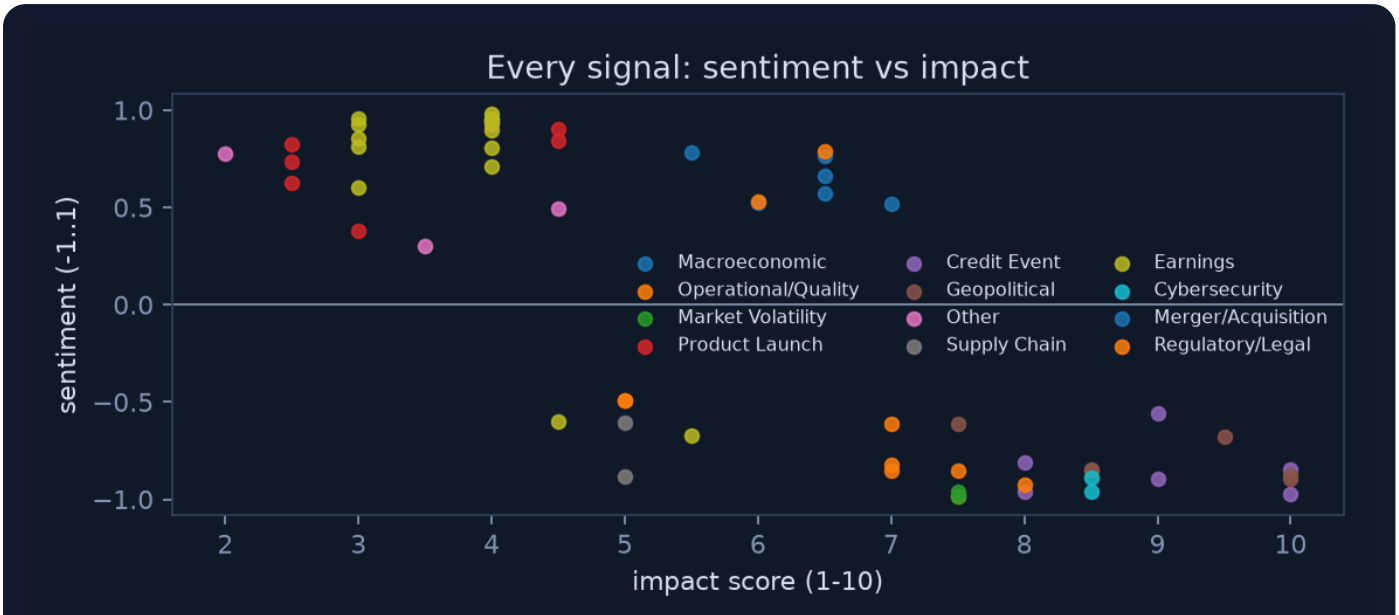
Position math (first-order, transparent): bonds/loans lose `notional × duration × Δrate` + `notional × Δspread`; loans take haircuts; pay-fixed swaps *gain* when rates rise; sold CDS loses when spreads widen; commitments take half-haircuts for drawdown risk.

Real results — 23 stress tests fired in the headless run:

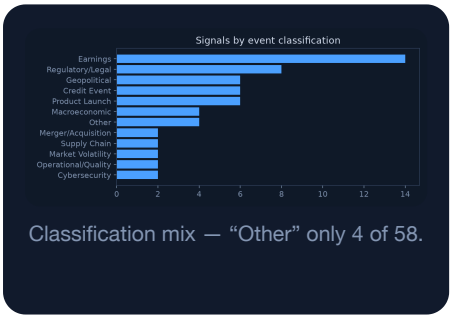
Scenario	Trigger (real signal)	Portfolio loss	Biggest single hit
Rates shock (+200bp)	"Fed… rate cut hopes reverse" class Macroeconomic, impact 9	**−$435m (−8.12%)**	30Y Treasury: −$92.5m (250 × 18.5 × 2%)
Credit shock	Pacific Crest Chapter 11, impact 10	**−$355m (−6.62%)**	HY bond + leveraged loan complex
Geo shock	OPEC+ surprise cut, impact 8.5	−$215m (−4.01%)	spreads +180bp across credit

7 Results — what the system actually produced

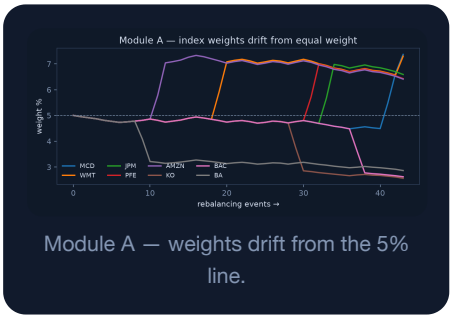
All figures below are real output from the headless end-to-end run (`python demo.py` — 58 items, both modules live).



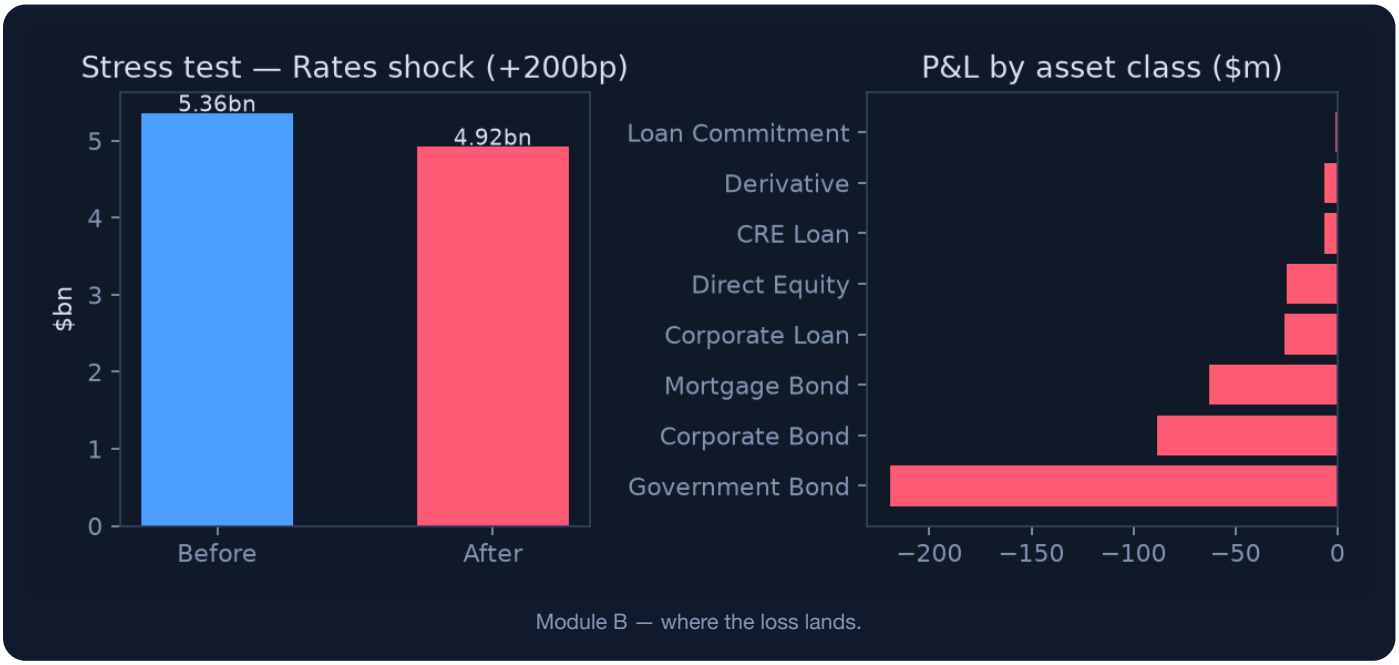
Every signal on one canvas — sentiment (y) vs impact (x), colored by event class. The engine separates the universe: high-impact negatives cluster top-right (Credit/Geo — the red zone), product/earnings news sits bottom-left.



Classification mix — "Other" only 4 of 58.



Module A — weights drift from the 5% line.



Module B — where the loss lands.

8 The 5-minute demo (exact script)

Setup: `python run.py --fast` → dashboard opens. Keep a terminal visible — it proves live processing.

Time	Where	What you say / do
0:00	Camera off / black	"Markets move on news, but risk systems only eat numbers. RiskRadar is the bridge — three numbers per headline: sentiment, event, impact. And two modules that act on them."
0:20	Engine tab	Point at the live table: new rows every few seconds. Read one aloud + its scores ("bankruptcy story: −0.97, Credit Event, impact 10, high-risk flag — and the engine shows *why*: matched keywords + impact drivers on every signal").
1:00	Engine tab → inject box	**The money shot.** Type a fresh headline no one has seen ("BREAKING: regulators open massive probe into chip shortage price fixing"), hit Analyze. Read the verdict, then point at the two chips: "Module A just rebalanced, Module B just triggered a stress test — one text in, two systems moved."
2:00	Sources line	"Requirement was two sources minimum — news + social both streaming here; live NewsAPI/GDELT/Reddit connectors activate when keys and internet exist. I fail soft: the demo never breaks."
2:20	Module A tab	Weights chart: "started 5% equal; Walmart/McDonald's floated up to ~7%, Boeing/BofA sank to ~2.6% on supply-chain and default news." Formula card + audit trail: "every weight change logs the signal that caused it."
3:20	Module B tab	"$5.36bn, 38 positions, nine asset classes. Impact ≥ 7 fires an event-typed scenario — last one was a credit event: −6.6%, and you can see it hurt loans and HY first." Worst-hit positions table.
4:20	Wrap	"One engine, three explainable numbers, two consumers — tactical rebalancing and strategic stress testing — running live, offline, on a laptop. Happy to take questions — or throw any headline at it."

Demo insurance: everything runs offline via the simulator. If Wi-Fi exists, run `python run.py` and let the connectors show live too — but never *depend* on them.

If something goes wrong mid-demo

- **Dashboard frozen?** The API is the same product: `curl localhost:8000/api/signals` — JSON stream is the point.
- **No browser at all?** `python demo.py` runs the full pipeline headless and prints the same numbers.

9 Challenges I hit — and how I solved them

The honest list. These make good “what was hard?” answers.

Problem	How I solved it
“Real-time” APIs kept failing me — no keys during development, rate limits, blocked networks	Fail-soft architecture: connectors are optional threads; a **LiveFeedSimulator** streams the same corpus with jitter so the rest of the system cannot tell the difference. The demo became unbreakable *because* live data was unreliable.
VADER was neutral on finance words — “downgrade”, “contagion”, “buyback” scored 0	Built a ~150-term finance lexicon by reading my own corpus and adding every jargon word VADER missed. (First sanity check: “bankruptcy” must score very negative. It does: –0.97 on the test headline.)
Classification conflicts — “bankruptcy amid lawsuit” matched both Credit and Legal rules	Priority ordering + most-matches-wins. Domain call: a Chapter 11 filing is a *Credit Event* even if the story also mentions litigation.
“Impact 7” had no definition — what does severity even mean with no historical data?	Made it a documented formula: base per category + transparent modifiers, every driver stored on the signal. I chose auditability over pretending to learn it from data I don’t have.
One stock dominating the mock index — early tests sent TSLA to 20%+	Position limits (2–12%) + renormalization — which is also how capped real indices behave, so the fix doubled as realism.
Stress test felt arbitrary — why these shocks?	Anchored scenarios to the case study’s own examples (rates +2%, equity –10%) and extended per event type; kept all position math first-order so every dollar of loss is traceable (duration × Δrate etc.).
Multi-line JSON feeds broke my JSONL loader	Wrote a tolerant loader (raw_decode loop) handling both strict JSONL and pretty-printed files — worth it: my bundled data stays human-readable and diff-friendly.
Dashboard needed a chart CDN — but the venue has no Wi-Fi	Bundled the entire chart library (plotly.min.js, ~4.8MB) into the repo. Zero network needed, ever.

10 Limitations & the upgrade path

Volunteering limitations before judges find them = credibility. Each row has a concrete next step.

Limitation (I know)	Why it exists	What I'd do next
Sentiment is lexicon-based, not learned	must run offline/instant/explainable for a live demo	drop-in **FinBERT** behind the existing provider interface; keep lexicon as fallback + explainability layer
Impact scores are domain-judged, not calibrated against history	no reliable event→market-impact dataset within hackathon time	calibrate bases against historical event studies / GDELT outcomes; log per-category distributions over time
Rules can't read sarcasm or rumors in tweets	that's genuinely hard NLP	source-credibility weighting + corroboration across sources before high flags; LLM second-opinion pass on high-impact items only
Entity matching = curated 20-name dictionary	false-positive-free within my universe, instant	spaCy NER + ticker-linking service beyond ~100 names
Module A has no turnover limits, transaction costs, or risk model	out of scope by design (mock index)	backtest on Kaggle tweet/price data (the case study's own dataset), add cost-aware optimization
Module B is first-order (no VaR, correlations, netting, wrong-way risk)	case study explicitly allows a simplified model	scenario VaR + at least bond-level netting; feed results into a proper risk engine later
Engine is a poll-based Flask app on one machine	right-sized for the problem	same pub/sub core behind Kafka/WebSockets scales horizontally; the interfaces wouldn't change
No auth on the API	demo tool	token auth + rate limits for any real deployment

What I deliberately did NOT fake

- No "trained model" claims — there is no hidden accuracy number I can't defend.
- No live-data pretense — bundled feeds are labeled as samples; live connectors are real but optional.
- No portfolio data provenance games — the portfolio is synthetic *by requirement* and I say so.

If you only remember one line about limitations: "I chose transparent-first over learned-first on purpose — for a risk tool, every number must survive the question 'why?'. The provider interface is exactly where ML plugs in next."

11 Q&A prep — design & engineering questions

Model answers, in the voice you'd actually speak in. Short versions live in the cheat sheet (last page).

Q1. "Why VADER and not FinBERT, an LLM, or a transformer?"

Three deliberate reasons. First, the demo must run live on any laptop with zero internet — a model download that fails on stage is a failed demo. Second, explainability: for a *risk* product, every score has to survive "why –0.97?" — with VADER plus my finance lexicon I can show the exact words that did it. Third, half my corpus is social media, and VADER was literally built for tweets — emoji, caps and negation work out of the box. FinBERT is a genuine upgrade and it's a one-file change behind my provider interface — that's on my roadmap, not my cutting-room floor.

Q2. "How accurate is the sentiment, really?"

On my 58-item corpus I sanity-checked every score by hand: 30 positive, 28 negative, zero sign flips versus my own reading — the bankruptcy headline hits –0.97, the earnings-beat +0.96. I'm not claiming SOTA accuracy; I'm claiming correctness on my domain sample plus full explainability. Proper validation against a labeled set like the Kaggle "Financial PhraseBank" is step one of the future work — that's an honest answer, and judges usually reward the honesty more than an inflated number.

Q3. "Why rule-based event classification instead of ML?"

Bounded domain, ~12 categories, no training labels needed, instant on CPU, and every signal stores the keywords that fired — a free audit trail. On my corpus, 54 of 58 items got a meaningful class (93%), and the 4 leftovers were casual chatter where "Other" with low impact is the *correct* answer. If the category set grew or the language got messier, I'd move to zero-shot classification — the interface wouldn't change.

Q4. "How did you calibrate the impact score? What does '7' even mean?"

It's a documented formula, not a learned number: a base severity per category (Credit 7, Geopolitical 6.5, Macro 6 … Product 3), plus modifiers for urgency ("breaking"), magnitude ("record"), strong negative sentiment, and broad scope. So a 7 means "a serious, multi-name, negative, category-typical event" — and because every driver is stored on the signal, anyone can audit the arithmetic. Calibrating the bases against historical event studies is on the roadmap; I chose transparent over pretending.

Q5. "What is 'real-time' about this, honestly?"

Processing latency is ~0.3 ms per item — about 3,300 signals per second on one laptop core. The dashboard polls every 2.5 seconds and sources stream continuously, so end-to-end you see news hit the charts in under three seconds. I don't pretend to be a microsecond trading system: the real-time constraint for news is seconds, and I'm comfortably inside it with headroom to scale.

Q6. "Why Flask? Why SQLite? Why both modules?"

Flask because eight endpoints don't justify FastAPI ceremony — the smallest tool that works is the most reliable on stage. SQLite + JSONL because the case study said "API *or* file" and I did both: SQLite for history queries, JSONL as the grep-able stream. Both modules because they showcase the same engine feeding opposite timescales — tactical (rebalance now) and strategic (stress the book) — and once the pub/sub core existed, Module B was ~150 lines. Cheap to build, expensive-looking to demo.

11 Q&A prep — finance & domain questions

Q8. “Would a real index team actually use your Module A?”

Not as-is — as an overlay. Real deployment would add turnover limits, transaction costs, an execution layer and a proper risk model; sentiment would be one input among many. What I built demonstrates the feedback loop that matters: news → signal → weight change, with position limits and an audit trail — the same shape a production version would have, at hackathon scale.

Q9. “Why k=0.25, and why the 2–12% band?”

k=0.25 means a single maximal-sentiment event moves a weight by 25% of its size — noticeable but not jumpy; compounding over a stream of signals is where the drift comes from. The band mirrors how real capped indices work (no single name dominates), and in early testing it also fixed a real bug for me: Tesla once hit 20%+ before I added limits. Constraint as realism.

Q10. “Is your stress test realistic?”

It’s first-order on purpose: bonds lose notional × duration × Δrate plus spread widening; loans take haircuts; pay-fixed swaps gain when rates rise. That’s the same mechanic as simplified regulatory scenario approaches — minus VaR, correlations and netting. What it does well is what a stress test is *for*: ranking vulnerability by size and showing where the loss lands. The case study itself says the model may be simplified; I kept every dollar traceable instead.

Q11. “Why did the rates shock hurt most (–8.1%)? That portfolio isn’t rate-heavy.”

Because duration compounds: a 30-year Treasury has 18.5 years of it, so a +200bp move costs 250 × 18.5 × 2% ≈ $92m on one position, and the whole bond book bleeds together — $435m total. The credit shock is next (–6.6%) because spreads hit bonds *and* loans *and* the sold CDS. That ordering is a nice validation: the math behaves like a rate-sensitive balance sheet should.

Q12. “What’s missing versus a real risk platform (Bloomberg PORT, Murex…)?”

Position-level netting, wrong-way risk, liquidity haircuts, VaR/ES, model validation, data lineage — all named in my limitations table. I built the layer that was in scope: event detection and first-order scenario P&L, done traceably. The pub/sub core is exactly where a real engine would subscribe.

Q13. “Why a wholesale banking portfolio?”

The case study asked for loans, bonds and derivatives mixed together — that’s a bank balance sheet. I synthesized 38 positions across nine asset classes with realistic fields (rating, duration, spread, direction) and deliberate concentration: e.g. Velocity Media appears as both a leveraged loan and a high-yield bond, so credit shocks bite twice — like real life.

Q14. “Your news is simulated. Is anything real?”

The engine, API, both modules, persistence and dashboard are fully real code you can run in one command. The *data* is two bundled feeds of realistic items because venue Wi-Fi can’t be trusted — and I say exactly that. Live connectors (NewsAPI, GDELT, Reddit) are also real; give them keys and internet and they activate. The safest demo is the one that can’t break.

11 Q&A prep — curveballs & process questions

Q15. “I have a headline right here — run it.”

Love that question. Paste it in the dashboard’s “Try it live” box or POST it to */api/inject*. The verdict appears in ~0.3 ms with full evidence — keywords, drivers, flag — and you’ll see the two chips say whether Module A rebalanced and Module B fired. Practicing two or three of these before the demo is the best prep you can do.

Q16. “How do you handle sarcasm, bots, or market rumors in tweets?”

Honestly: not fully — that’s hard NLP and I won’t pretend otherwise. But the design dampens it: impact needs category + sentiment + scope to align, a single casual tweet rarely clears the bases needed for a high score, and high-risk flags prefer multi-evidence items. A production version would add source-credibility weighting and require corroboration before alerting. I kept the failure mode boring rather than invisible.

Q17. “‘Apple’ could mean fruit. How does entity matching not break?”

Word-boundary matching means “apple pie” won’t hit “Apple”, and the alias list maps brands to tickers (Google → GOOGL). With a curated 20-name universe, false positives are near zero — that’s a luxury of the mock-index scope, and it’s why I’d switch to spaCy NER only if the universe grew past a few hundred names.

Q18. “What happens if a live API dies mid-demo?”

Nothing visible. Connectors are fail-soft threads: no key, no network, or a dead endpoint just logs “inactive” and the simulator keeps the pipeline full. I demo with *zero* network dependency on purpose.

Q19. “How would you scale this to production?”

Horizontally on the same shape: sources → queue (Kafka) → engine workers (stateless, so N cores = N× throughput) → signal bus → consumers. The pub/sub interfaces I already have are the load-bearing walls; swapping Flask for a WSGI server behind a load balancer is configuration, not redesign. The NLP itself is the easy part to upgrade via the provider interface.

Q20. “What would you do with one more week?”

FinBERT sentiment behind the provider interface; backtest Module A on the Kaggle tweet/price datasets the case study recommends; add scenario VaR to Module B; source-credibility weighting for social; and Docker + CI so it runs anywhere. Each is already isolated to one file because of how I cut the interfaces.

Q21. “What was hardest?”

Defining what “impact 7” means with no historical dataset — I solved it by making the formula transparent and storing its drivers on every signal. Right behind it: classification priority (bankruptcy-plus-lawsuit should file as Credit, not Legal) and making the whole thing demo-proof.

Q22. “Did you use AI tools to build this?”

“Yes — for boilerplate and debugging, like nearly everyone here. The architecture, the scoring design, the trade-offs in this report — those are my decisions, and I can defend every one of them: pick any file and I’ll walk you through why it’s shaped that way.” (Then actually be ready to do that — the cheat sheet overleaf is the minimum.)

12 Cheat sheet — 10 numbers to know cold

CORPUS
58
28 news + 30 social · 93% classified (54) · 4 “Other” · 22 high-risk flags

LATENCY
0.3 ms
median per signal · ~3,300/sec on one core · dashboard polls 2.5 s

SOURCES
2 + 3
news + social bundled feeds · NewsAPI, GDELT, Reddit connectors (fail-soft)

MODULE A RULE
×(1+0.25s)
clamp 2–12% · renormalize · 20 stocks · 45 weight changes in demo run

MODULE A EXTREMES
7.4% / 2.6%
MCD top · KO & BAC bottom · started equal at 5.00%

MODULE B PORTFOLIO
$5.36bn
38 positions · 9 asset classes · corporate loans $1.6bn is the biggest bucket

STRESS TRIGGER
≥ 7
impact threshold (case study’s own number) · 23 tests fired in demo run

WORST STRESS
–8.1%
rates shock +200bp = –$435m · 30Y Treasury alone –$92.5m · credit shock –$355m

IMPACT FORMULA
base ± Δ
Credit 7 · Geo 6.5 · Macro 6 · … · Product 3 · modifiers: urgency/magnitude/sentiment/scope

Six one-liners worth memorizing

- “Two source types were required — mine has two feed types and three live connectors, fail-soft by design.”
- “Three numbers out of any text: sentiment, event, impact — each one carries its own evidence.”
- “Explainability was a requirement, not a nice-to-have — this is a risk tool.”
- “Every trade-off in this project is written down in the report — ask me about any of them.”

Files to point at when a judge asks to see code

If they ask about…	Open…
The core pipeline	`engine/pipeline.py`
Sentiment / finance lexicon	`engine/lexicon.py`
Classification rules & priorities	`engine/classifier.py`
Impact formula	`engine/impact.py`
Rebalancing rule	`modules/rebalancer.py`
Stress scenarios & math	`modules/stress_test.py`